



U N I V E R S I D A D
COMPLUTENSE
M A D R I D



Proyecto: optimización del *rendezvous* para dos *cobots* cooperativos

Máster en Ingeniería de Sistemas y Control

Asignatura: Optimización Heurística y Aplicaciones

Curso: 2025/2026

Alumno: Javier Arambarri Calvo

8 de junio de 2026



Autoría

El autor de este documento es Javier Arambarri Calvo, alumno del Máster en Ingeniería de Sistemas y Control y matriculado por la Universidad Complutense de Madrid en el curso académico 2025-2026.

Para que así conste, se firma digitalmente este documento.



Índice

Introducción	4
Tarea	5
Propuesta: optimización del <i>rendezvous</i> para dos <i>cobots</i> cooperativos	6
Variables de decisión	8
Función objetivo	8
Restricciones	9
Función de mérito penalizada	10
Notas	11
Propuesta técnica de búsqueda local	13
Algoritmo elegido	13
Diseño del algoritmo	13
Configuración de parámetros	14
Diseño experimental	14
Resultados (esquema)	14
Conclusiones (expectativas)	15
Revisión y desarrollo de la propuesta técnica de búsqueda local y propuesta final	16
Revisión del profesorado	16
Diseño de la búsqueda de la solución	16
Diseño del algoritmo	17
Configuración de parámetros	20
Diseño experimental	21
Resultados	22
Conclusiones	26
Propuesta técnica de búsqueda evolutiva	27
Desarrollo de la propuesta técnica de búsqueda evolutiva	29
Resultados principales	29
Conclusiones	33
Definición multi-objetivo del problema y resolución	35
Referencias adicionales al Campus Virtual	37



Introducción

Esta memoria recoge el trabajo realizado en la asignatura de Optimización Heurística y Aplicaciones del Máster en Ingeniería de Sistemas y de Control.

A lo largo de la asignatura se han ido recibiendo correcciones por parte del equipo docente que se han subsanado.

Asimismo, en el caso de la implementación de la técnica de búsqueda local se han identificado diversas dificultades que han requerido la modificación de la definición del problema. Por ello, estas modificaciones se han ido comentando según se resolvía el problema y se ha mantenido la versión inicial en las secciones correspondientes con el fin de reflejar la evolución sufrida.

Este documento se organiza de la siguiente manera: en la sección Tarea se describe la tarea de la asignatura, en la sección de la Propuesta se define la problemática a resolver, en la sección de la Propuesta Técnica de Búsqueda Local se presenta la propuesta inicial realizada en el tema 2 y por último, en la sección de Revisión y Desarrollo de la Propuesta Técnica de Búsqueda Local se actualiza la propuesta inicial, se aplican las modificaciones oportunas para conseguir resolver el problema y se exponen los resultados obtenidos.



Tarea

Como uno de los objetivos fundamentales de la asignatura es conseguir que los alumnos sean capaces de enfrentarse a un problema de optimización real con las técnicas aprendidas, el primer trabajo que se propone consiste en que cada alumno sugiera el problema de optimización que desee resolver a lo largo de la asignatura, identifique sus variables de decisión, funciones objetivo, restricciones, grupo al que pertenecen, etc.

Con el objeto de nivelar los problemas elegidos por los alumnos, los problemas elegidos serán propuestos por los alumnos a los profesores de la asignatura, y analizados, modificados, refinados, y finalmente aceptados o rechazados por los últimos hasta alcanzar una propuesta formal del problema que los alumnos irán resolviendo a lo largo de la asignatura.

Este proceso de definición y selección de problemas se realizará durante el estudio de los contenidos de este primer tema. Debido al trabajo en equipo que se establece entre los profesores y los alumnos, se proponen dos fechas finales de entrega de la definición del problema:

- 15 de Marzo: Fecha límite de entrega de la propuesta inicial del problema en la que el alumno presentará un documento que describa (cualitativamente) las características más relevantes del problema que desea resolver, identificando las variables de decisión y las funciones de mérito utilizadas para medir los objetivos y restricciones del problema.
- Durante las dos semanas sucesivas, los profesores y el alumno analizarán el problema, lo modificarán, etc... con el objeto de acordar una propuesta inicial definitiva antes del 29 de Marzo. Para esta fecha, el alumno documentará la propuesta definitiva a los profesores, poniendo de manifiesto las variables de decisión, funciones de mérito, restricciones, clasificación del problema, etc... Además, implementará las funciones responsables de medir la bondad (mérito y restricciones) de una posible solución del problema. En las sesiones de dudas del tema (Semana 23 - 27 Febrero) se propondrá a los alumnos la cabecera de la función, de forma que sea utilizada, sin cambios significativos, durante los siguientes temas.

Propuesta: optimización del *rendezvous* para dos *cobots* cooperativos

En el grupo de investigación en el que trabajo estamos comenzando un proyecto enmarcado en la agricultura vertical. En este sentido, estamos preparando un entorno, tanto real como simulado (Figura 1), en el que dos robot manipuladores colaborativos van a desempeñar diferentes tareas. Debido a que todavía estamos definiendo el escenario, lo más probable es que este varíe respecto de la imagen presentada, aunque esas modificaciones no serán estructurales. Dos de los cambios incluyen que el robot de la izquierda tendrá también sus propias estanterías y que el robot de la derecha trabajará cerca de la pared cuando el eje lineal de desplazamiento sobre el que se coloca esté en uno de sus extremos.

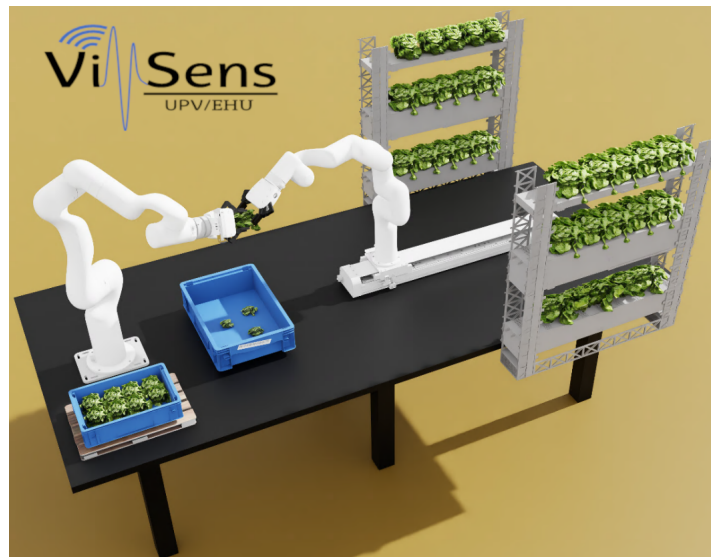


Figura 1: Escenario simulado.

Este escenario está compuesto por una mesa, dos robots manipuladores colaborativos y estanterías de cultivo y almacenaje. La mesa es la superficie sobre la que se desarrollará toda la actividad, por lo que los robots no deben exceder sus límites. El primer robot, el *UFactory 850* [1], es el que está colocado a la izquierda en la Figura 1, y está fijo a la mesa. El segundo robot, el *UFactory xArm6* [1], es el que se encuentra a la derecha en la Figura 1, y está montado sobre un eje lineal de desplazamiento que le permite aumentar su espacio de trabajo. Ambos robots están equipados con pinza, cámara de visión y sensor de fuerza en el efector. Las estanterías correspondientes a cada robot están colocadas de manera que son completamente alcanzables por los dos manipuladores.

El segundo robot se encargará de supervisar el estado de los cultivos o aplicarles alguna operación, y cuando considere necesario (bien sea por visión, o por tiempo de cultivo, por ejemplo) retirará un cultivo para realizarle una operación o tratamiento que necesite de la cooperación con el primer robot, por ejemplo, la retirada de hojas enfermas. En función del tratamiento aplicado, la hortaliza deberá volver a su estantería de cultivo, mediante el segundo

robot, o ir a la estantería de almacenaje. En caso de ser almacenada, será el primer robot el encargado de ejecutar dicha tarea. Por su parte, el primer robot traslada las hortalizas de la estantería de almacenaje a un palet o realiza alguna operación específica sobre las plantas en este momento del cultivo.

Además, en las tareas cooperativas será preciso controlar la fuerza aplicada por cada robot para evitar dañar las hortalizas, que se caracterizan por ser deformables. Siempre y cuando esta fuerza se controle adecuadamente por ambos robots, el punto de cooperación de los robots no tiene por qué ser fijo, esto es, en función de la situación instantánea de cada robot ese punto podrá darse en diferentes coordenadas, pero siempre garantizando que la fuerza aplicada a la planta es la adecuada.

Con todo ello, se busca un sistema autónomo y adaptativo en el que los dos robots decidan en cada instante qué operación ejecutar —retirar hojas, realizar tratamientos, regar, germinar o mover plantas al almacén, por ejemplo— de modo que el conjunto minimice el tiempo de ciclo, o, en otras palabras, maximice la eficiencia operativa.

Como se ha explicado, este sistema está compuesto de múltiples partes, pero esta propuesta de problema de optimización se va a centrar en el punto espacial de cooperación. A este punto se le denomina *rendezvous*.

Objetivo

Obtener las posiciones, $\mathbf{p}_r = (x_r, y_r, z_r)$, y orientaciones, $\mathbf{q}_r = (q_{r0}, q_{r1}, q_{r2}, q_{r3})$, de los efectores de cada robot, $r \in \{A, B\}$, y los factores de velocidad de ejecución de movimiento de cada robot, $\mathbf{s}_r$, incluyendo el del eje lineal de desplazamiento $\mathbf{s}_{rail}$, que permitan sincronizar la llegada de ambos robots al *rendezvous* óptimo calculado (p_r, q_r) sincronizando y minimizando el tiempo de desplazamiento necesario, $t_r(\mathbf{p}_r, \mathbf{q}_r, \mathbf{s}_r)$, según el estado actual de cada robot (posición actual).

Las posiciones se definen en el espacio de la tarea de cada robot y en metros, las orientaciones en cuaterniones, los factores de velocidad podrán tomar cualquier valor entre 0 y 1, y el tiempo de desplazamiento se mide en segundos. La elección de cuaterniones para representar la orientación no es trivial, dado que es la fórmula computacionalmente más eficiente [2] al no presentar singularidades, ser más compactos y fáciles de normalizar.

Por tanto, se buscan las posiciones y orientaciones de los efectores $\mathbf{p}_A = (x_A, y_A, z_A)$, $\mathbf{p}_B = (x_B, y_B, z_B)$, $\mathbf{q}_A = (q_{A0}, q_{A1}, q_{A2}, q_{A3})$, $\mathbf{q}_B = (q_{B0}, q_{B1}, q_{B2}, q_{B3})$, y los factores de velocidad de cada robot $\mathbf{s}_A, \mathbf{s}_B, \mathbf{s}_{rail}$, tal que minimicen el tiempo de desplazamiento de cada robot al punto de cooperación, $t_A(\mathbf{p}_A, \mathbf{q}_A, \mathbf{s}_A)$ y $t_B(\mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_B, \mathbf{s}_{rail})$, y ambos sean lo más parecidos posible.

Es decir, ambos robots deben llegar sincronizados al *rendezvous*. Es importante mencionar que, puesto que el segundo robot B está sobre un eje lineal de desplazamiento, su tiempo t_B tiene que tener en cuenta también el factor de velocidad de dicho eje, $\mathbf{s}_{rail}$.

En el punto de cooperación o *rendezvous* los efectores deben quedar enfrentados, esto es, alineados en dos ejes del mundo y separados por una determinada distancia en el otro. Esta configuración, Figura 2, implica que la diferencia en la orientación en ambos será de 180° y la distancia se medirá en línea recta desde los orígenes de ambos efectores, para lo que será necesario convertir las posiciones p_A y p_B a coordenadas del mundo.

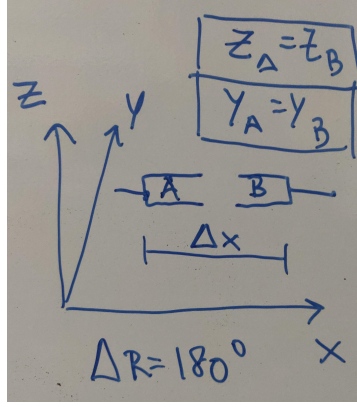


Figura 2: Croquis de la configuración propuesta.

En otras palabras, el objetivo es obtener las coordenadas óptimas del punto de cooperación *rendezvous* y los factores de velocidad de cada robot de manera que se minimice el tiempo de llegada sincronizada al mismo.

Variables de decisión

Se tiene un total de 17 variables de decisión:

- **Posiciones objetivo de efectores:** $\mathbf{p}_A = (x_A, y_A, z_A)$, $\mathbf{p}_B = (x_B, y_B, z_B)$ en el espacio de la tarea de cada robot.
- **Orientaciones objetivo de efectores:** $\mathbf{q}_A, \mathbf{q}_B \in S^3$, donde cada cuaternión unitario $\mathbf{q} = (q_0, q_1, q_2, q_3)$ representa una rotación en $SO(3)$.
- **Escalas de velocidad:** $s_A, s_B, s_{rail} \in (0, 1]$.

Función objetivo

$$\min_{\mathbf{p}_A, \mathbf{q}_A, \mathbf{p}_B, \mathbf{q}_B, s_A, s_B, s_{rail}} T_{rendezvous} = \min \max (t_A(\mathbf{p}_A, \mathbf{q}_A, s_A), t_B(\mathbf{p}_B, \mathbf{q}_B, s_B, s_{rail})),$$

donde los tiempos t_A y t_B se refiere a los tiempos $t_r(\mathbf{p}_r, \mathbf{q}_r, s_r)$ necesarios por cada robot para alcanzar el punto de cooperación. Se estima de la siguiente manera:

$$t_r(\mathbf{p}_r, \mathbf{q}_r, s_r) = \frac{\text{dist_rail}(\mathbf{p}_r)}{s_{rail} v_{rail}} + \frac{\ell_r(\mathbf{p}_r, \mathbf{q}_r)}{s_r v_{arm,r}},$$

En esta expresión $\text{dist_rail}(\mathbf{p}_r)$ es la distancia a recorrer por el motor lineal (raíl) en metros (si no existe raíl, es decir, el robot está fijo -robot A-, se toma 0), v_{rail} es la velocidad del raíl en m/s, $\ell_r(\mathbf{p}_r, \mathbf{q}_r)$ es la longitud efectiva del movimiento del brazo en metros (por ejemplo, la distancia euclídea del efector desde su posición inicial con todas las articulaciones en posición 0), y $v_{arm,r}$ es la velocidad del brazo en m/s. Dividir por s_r reduce la velocidad efectiva y por tanto, aumenta el tiempo.

Cabe mencionar que la primera evaluación del equipo docente valoró la sustitución de la función $\min \max(\cdot)$ por un frente de pareto multiobjetivo con dos funciones objetivo, una por cada robot:

$$\min_{\mathbf{p}_A, \mathbf{q}_A, \mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_A, \mathbf{s}_B, \mathbf{s}_{rail}} \mathbf{T}(\cdot),$$

con

$$\mathbf{T}(\cdot) = \begin{bmatrix} t_A(\mathbf{p}_A, \mathbf{q}_A, \mathbf{s}_A) \\ t_B(\mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_B, \mathbf{s}_{rail}) \end{bmatrix},$$

Así, una solución $\mathbf{x}^*$ se considera óptima en el sentido de Pareto si no existe otra solución factible $\mathbf{x}$ tal que

$$t_A(\mathbf{x}) \leq t_A(\mathbf{x}^*), \quad t_B(\mathbf{x}) \leq t_B(\mathbf{x}^*),$$

y al menos una de las desigualdades sea estricta ($<$). El conjunto de todas las soluciones Pareto-óptimas constituye el frente de Pareto en el plano (t_A, t_B) , que describe los compromisos entre minimizar el tiempo de llegada del robot A y el del robot B.

Mientras que con Pareto no hay una única solución y se necesitaría un criterio externo para elegir un punto de entre los Pareto-óptimos, la formulación con $\min \max(\cdot)$ obtiene una única solución. Puesto que se desea que ambos robots lleguen sincronizados, en otras palabras, que tarden lo mismo, se va a optar por $\min \max(\cdot)$, aunque se deja la puerta abierta a resolver también el problema como frente de Pareto y así desarrollar una comparativa.

Restricciones

- Separación final en coordenadas del mundo. Los efectores deben quedar separados por la distancia objetivo d_{ef} en el eje asignado, cuyo eje unitario se representa mediante $\mathbf{e}_{eje}^\top$, y con tolerancia ϵ_{eje} : $|\mathbf{e}_{eje}^\top((\mathbf{p}_A) - (\mathbf{p}_B)) - d_{ef}| \leq \epsilon_{eje}$.
- Posición y orientación de enfrentamiento: *rendezvous* enfrentado. Siguiendo la representación de la Figura 2:

$$|\mathbf{e}_Z^\top((\mathbf{p}_A) - (\mathbf{p}_B))| \leq \epsilon_Z, \quad |\mathbf{e}_Y^\top((\mathbf{p}_A) - (\mathbf{p}_B))| \leq \epsilon_Y$$

$$\|\text{Log}(\mathbf{R}_B \mathbf{R}_A^\top \mathbf{R}_\pi^\top)\| \leq \epsilon_\theta,$$

donde $\mathbf{R}_\pi$ es la rotación de 180° que define la orientación volteada y $\text{Log}(\cdot)$ es el mapa logaritmo de rotaciones (norma en radianes).

Aunque las orientaciones se parametrizan mediante cuaterniones en las variables de decisión, para evaluar las restricciones resulta conveniente convertirlos a matrices de rotación $\mathbf{R}_r$. La representación en $SO(3)$ permite operar directamente con la composición de rotaciones y, en particular, aplicar el mapa logarítmico $\text{Log}(\cdot)$, que transforma una matriz de rotación en su representación eje-ángulo [3]. La norma de este vector proporciona una medida angular interpretable y numéricamente estable del error entre orientaciones, equivalente al ángulo mínimo necesario para alinear una con la otra [4]. Por ello, la restricción $\|\text{Log}(\mathbf{R}_B \mathbf{R}_A^\top \mathbf{R}_\pi^\top)\| \leq \epsilon_\theta$ cuantifica de manera directa la desviación respecto a la orientación enfrentada deseada. De este modo, las restricciones de alineación y enfrentamiento pueden formularse de forma más natural y robusta, manteniendo al mismo tiempo la compacidad y suavidad de los cuaterniones como parametrización principal en la optimización (https://cvgl.cit.tum.de/_media/members/demmeln/nurlanov2021so3log.pdf, <https://colab.research.google.com/github/borglab/gtsam/blob/develop/gtsam/geometry/doc/S03.ipynb>).

Esta restricción, aparentemente dura, se ha diseñado como blanda puesto que el objetivo a largo plazo será que el *rendezvous* se pueda alcanzar con diferentes orientaciones y alineaciones. No obstante, para simplificar el proyecto dentro de la asignatura, se introduce la restricción.

- Sincronización temporal. Llegadas casi simultáneas dentro de la tolerancia $\Delta t_{\text{máx}}$: $|t_A(\mathbf{p}_A, \mathbf{q}_A, \mathbf{s}_A) - t_B(\mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_B, \mathbf{s}_{\text{rail}})| \leq \Delta t_{\text{máx}}$.
- Límites de escala de velocidad: $0 < s_A \leq 1$, $0 < s_B \leq 1$, siendo 1 la velocidad máxima permitida por el robot. Esta restricción será por definición del problema y no una restricción pura a validar por el algoritmo.
- Exclusión de singularidades y factibilidad cinemática. Para cada pose y orientación objetivo debe existir una solución de cinemática inversa q_r alcanzable:
 $\exists q_r : FK_r(q_r) = (\mathbf{p}_r, \mathbf{R}_r), \quad \sigma_{\min}(J_r(q_r)) \geq m_{\min}$, donde $\sigma_{\min}(J_r(q_r))$ es el menor valor singular de la jacobiana en q_r , es decir, una medida numérica de cuán cerca está la configuración de una singularidad. Valores pequeños indican pérdida de grado de libertad efectivo, es decir, que se encuentra cerca de una singularidad. Si existen múltiples soluciones IK (cinemática inversa), se selecciona la que maximice $\sigma_{\min}$.
- Espacio de trabajo $\mathcal{W}_r$ y zonas prohibidas $\mathcal{Z}_r$ de cada robot r . Las poses deben pertenecer al espacio de trabajo y evitar zonas prohibidas: $(\mathbf{p}_r, \mathbf{R}_r) \in \mathcal{W}_r \setminus \mathcal{Z}_r$, $r \in \{A, B\}$. Esta restricción debería ser dura porque si no se cumple la solución es directamente no factible. No obstante, esta restricción también irá ligada a la definición del problema y no será una restricción pura a validar por el algoritmo.

Función de mérito penalizada

En primera instancia se propuso una función de mérito que combinaba el objetivo principal, $T_{\text{máx}}(\cdot) = \max(t_A(\cdot), t_B(\cdot))$, donde $\cdot$ denota el conjunto de variables de decisión, con las siguientes penalizaciones por violación de restricciones blandas:

- **Penalización de separación:** penaliza desviaciones mayores que ϵ respecto a la distancia objetivo d_{ef} proyectada sobre el eje.

$$P_{\text{sep}}(\mathbf{p}_A, \mathbf{p}_B) = \left(\max(0, |\text{proj}_{\text{eje}}(\mathbf{p}_A) - \text{proj}_{\text{eje}}(\mathbf{p}_B) - d_{\text{ef}}| - \epsilon) \right)^2,$$

- **Penalización de alineación:** penaliza desviaciones en la alineación de los efectores en el *rendezvous*.

$$\begin{aligned} P_{\text{alineación}}(\mathbf{p}_A, \mathbf{p}_B, \mathbf{R}_A, \mathbf{R}_B) = & \left(\max(0, |\mathbf{e}_Z^\top(\mathbf{p}_A - \mathbf{p}_B)| - \epsilon_Z) \right)^2 \\ & + \left(\max(0, |\mathbf{e}_Y^\top(\mathbf{p}_A - \mathbf{p}_B)| - \epsilon_Y) \right)^2 \\ & + \left(\max(0, \|\text{Log}(\mathbf{R}_B \mathbf{R}_A^\top \mathbf{R}_\pi^\top)\| - \epsilon_\theta) \right)^2. \end{aligned}$$

- **Penalización de sincronización:** penaliza desincronizaciones mayores que la tolerancia $\Delta t_{\text{máx}}$.

$$P_{\text{sync}}(t_A, t_B) = \left(\max(0, |t_A - t_B| - \Delta t_{\text{máx}}) \right)^2,$$

- **Penalización de singularidades:** penaliza configuraciones con manipulabilidad por debajo del umbral $m_{\min}$; $\sigma_{\min}(J_r)$ es el menor valor singular del Jacobiano.

$$P_{\text{sing}}(\mathbf{p}_r, \mathbf{R}_r) = \sum_{r \in \{A, B\}} \left(\max(0, m_{\min} - \sigma_{\min}(J_r(q_r(\mathbf{p}_r, \mathbf{R}_r)))) \right)^2,$$

Las penalizaciones se definen como cuadráticas porque castigan más fuertemente las violaciones grandes que las pequeñas, lo que incentiva al optimizador a priorizar correcciones moderadas antes que soluciones extremas.

La función de mérito quedaba de la siguiente manera:

$$J_{\text{total}}(\cdot) = T_{\text{máx}}(\cdot) + P_{\text{sep}}(\mathbf{p}_A, \mathbf{p}_B) + P_{\text{alineación}}(\mathbf{p}_A, \mathbf{p}_B, \mathbf{R}_A, \mathbf{R}_B) + P_{\text{sync}}(t_A, t_B) + P_{\text{sing}}(\mathbf{p}_r, \mathbf{R}_r).$$

Sin embargo, tras la primera evaluación del equipo docente se ha decidido suprimir la presencia de la función objetivo en la función de mérito. Esto se debe a que sería necesario introducir ponderaciones de difícil sintonización con el riesgo de introducir posibles discontinuidades asociadas al operador $\max(\cdot)$.

Se va a ilustrar mediante un ejemplo. Sean dos soluciones distintas para las variables de decisión donde la primera conlleva un menor tiempo de desplazamiento, pero no es tan precisa como la segunda. En estos casos, puede que nos interese más la segunda solución, pero introducir la función objetivo en la de mérito podría provocar que se desechara.

Consecuentemente, la función de mérito propuesta es la siguiente:

$$J_{\text{total}}(\cdot) = P_{\text{sep}}(\mathbf{p}_A, \mathbf{p}_B) + P_{\text{alineación}}(\mathbf{p}_A, \mathbf{p}_B, \mathbf{R}_A, \mathbf{R}_B) + P_{\text{sync}}(t_A, t_B) + P_{\text{sing}}(\mathbf{p}_r, \mathbf{R}_r).$$

Notas

La restricción sobre la exclusión de singularidades puede omitirse en una primera fase para simplificar el problema, y una vez resuelto, añadirla para ampliar el problema de manera escalonada.

Las restricciones de los factores de velocidad y de los espacios de trabajo se establecen como definición del problema. De esta manera, el algoritmo siempre generará nuevas soluciones dentro de las cotas establecidas en la naturaleza del problema. Por ejemplo, resulta evidente que las velocidades solo podrán estar en el rango $[0, 100]$ %. Por tanto, no se le permite generar una solución con un valor fuera de ese rango facilitando así la búsqueda de soluciones.

Todas las restricciones posicionales y orientacionales se evalúan en coordenadas del mundo: las poses objetivo se transforman desde la base del robot al mundo antes de aplicar las desigualdades anteriores.

La verificación de colisiones entre los robots durante las trayectorias que les permiten desplazarse desde la posición y orientación actual al *rendezvous* óptimo calculado queda fuera del alcance de este problema de optimización. Esto se debe a que se considera un problema del nivel de la planificación de trayectorias.

La principal línea de trabajo del problema es su resolución mediante la función objetivo mín máx($\cdot$). Asimismo, se contempla la posibilidad de abordar el problema desde una perspectiva multiobjetivo, obteniendo el correspondiente frente de Pareto, para realizar una comparativa entre ambos diseños.

Propuesta técnica de búsqueda local

Algoritmo elegido

Se ha seleccionado el algoritmo de Temple Simulado (*Simulated Annealing*, SA) dado que el espacio de búsqueda es continuo y existen restricciones geométricas y cinemáticas no triviales. Las variables incluyen posiciones en $\mathbb{R}^3$, orientaciones en forma de cuaterniones unitarios y factores de velocidad acotados, lo que dificulta la definición de vecindarios discretos o estructuras de memoria como las empleadas en Búsqueda Tabú. En cambio, El Temple Simulado permite explorar este espacio continuo mediante perturbaciones suaves y controladas.

Diseño del algoritmo

Representación de las soluciones

Cada solución candidata se representa como un vector continuo:

$$\mathbf{x} = (\mathbf{p}_A, \mathbf{p}_B, \mathbf{q}_A, \mathbf{q}_B, s_A, s_B, s_{rail}) \in \mathbb{R}^{17},$$

donde $\mathbf{p}_A, \mathbf{p}_B \in \mathbb{R}^3$ son las posiciones objetivo de cada robot, $\mathbf{q}_A, \mathbf{q}_B \in S^3$ son cuaterniones unitarios que representan orientaciones de cada efector y $s_A, s_B, s_{rail} \in (0, 1]$ son los factores de velocidad de cada robot y del motor lineal de desplazamiento.

Operadores de vecindad

El operador de vecindad genera una solución cercana mediante una perturbación gaussiana, $\mathbf{x}' = \mathbf{x} + \mathcal{N}(0, \sigma^2)$, pero adaptada a la naturaleza de cada tipo de variable:

- **Posiciones:** $\mathbf{p}'_r = \mathbf{p}_r + \mathcal{N}(0, \sigma_p^2)$, seguidas de una proyección al espacio de trabajo mediante truncamiento si alguna coordenada sale de los límites. Esta operación asegura que $\mathbf{p}'_r \in \mathcal{W}_r$ tras la mutación: $p'_{r,i} \leftarrow \min(p_{i,\max}, \max(p_{i,\min}, p'_{r,i}))$, $i \in \{x, y, z\}$.
- **Orientaciones:** $\mathbf{q}'_r = \mathbf{q}_r + \mathcal{N}(0, \sigma_q^2)$, donde el cuaternión resultante se vuelve a normalizar para garantizar que es unitario: $\mathbf{q}'_r \leftarrow \frac{\mathbf{q}'_r}{\|\mathbf{q}'_r\|}$.
- **Velocidades:** $s'_r = s_r + \mathcal{N}(0, \sigma_s^2)$, aplicando truncamiento para mantener la factibilidad física. $s'_r \leftarrow \min(1, \max(0, s'_r))$.

Gestión de restricciones

Los operadores de vecindad garantizan únicamente la factibilidad básica de las variables (*workspace*, cuaterniones unitarios y velocidades en $[0, 1]$). El resto de restricciones del problema se gestionan en la fase de evaluación de la solución:

- **Separación y alineación:** las penalizaciones P_{sep} y $P_{\text{alineación}}$ cuantifican la desviación respecto a la configuración de *rendezvous* deseada.
- **Sincronización temporal:** se penaliza mediante P_{sync} .

- **Factibilidad cinemática y singularidades:** Para cada pose candidata se verifica la existencia de una solución de cinemática inversa. Si no existe, la solución se descarta o penaliza mediante P_{sing} .

De este modo, el Temple Simulado explora libremente el espacio de búsqueda, mientras que la función de evaluación guía la convergencia hacia soluciones que cumplen las restricciones.

Configuración de parámetros

Se propone comenzar con una temperatura de $T_0 = 1$, un factor de enfriamiento $\alpha = 0,95$, un máximo de $L = 50$ movimientos o generación de vecinos por temperatura, un máximo de $K = 200$ temperaturas y una desviación del operador de vecindad de $\sigma = 0,1$. Esta configuración permite explorar $K \cdot L = 200 \cdot 50 = 10,000$ soluciones diferentes.

Diseño experimental

El plan de experimentación constará de 30 ejecuciones independientes cada una con una semilla aleatoria distinta, de tal forma que se garantice la validez estadística.

Las soluciones se evaluarán mediante tres métricas: calidad, tiempo de ejecución y número de evaluaciones. La calidad registra el mejor y peor valor final de $T_{\text{máx}}$, la media, la desviación típica y el intervalo de confianza. El tiempo de ejecución mide el tiempo total de *CPU* y se considera con la calidad de la solución lograda. Esta consideración se realizará a través de un diagrama de dispersión, como el de la Figura 3, y un valor de eficiencia calculado: $E = \frac{T_{\text{max}}}{\text{tiempo}}$. Por último, el número de evaluaciones anota la iteración en la que se alcanzó el mejor valor.

Además, dado que el problema incluye restricciones que pueden invalidar una solución, se registrará también la factibilidad de cada solución. A partir de ello se calculará el porcentaje de soluciones factibles obtenidas por el algoritmo.

Resultados (esquema)

Se presentan las Tablas 1 y 3 y la Figura 3 que se pretenden obtener para evaluar los resultados del algoritmo.

Ejecución	$T_{\text{máx}}$ final	Tiempo (s)	Iteración óptima	Eficiencia	% soluciones factibles
1	–	–	–	–	–
2	–	–	–	–	–
3	–	–	–	–	–
⋮	⋮	⋮	⋮	⋮	⋮
30	–	–	–	–	–

Tabla 1: Resultados individuales de las 30 ejecuciones del algoritmo.

Métrica	Valor
Media de $T_{\text{máx}}$	—
Desviación típica	—
Mejor valor (mínimo)	—
Peor valor (máximo)	—
Intervalo de confianza 95 %	—
Tiempo medio de ejecución (s)	—
% soluciones factibles	—

Tabla 2: Métricas agregadas calculadas sobre las 30 ejecuciones.

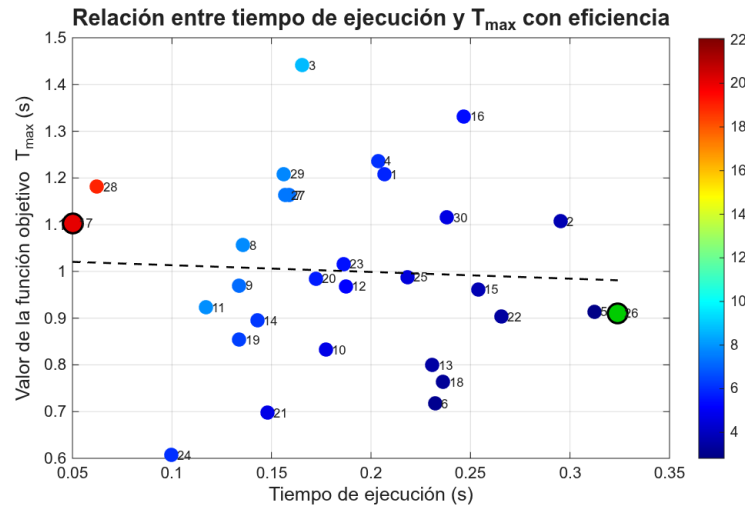


Figura 3: Diagrama de dispersión propuesto para la evaluación de la eficiencia de las soluciones. Cada círculo representa una solución cuyo número adjunto corresponde con el número de ejecución, el color de las soluciones se refiere a la eficiencia de las mismas, los círculos con borde negro representan las peor y mejor solución, en color rojo y verde, respectivamente, y la recta punteada la línea de tendencia.

Conclusiones (expectativas)

Se espera que el Temple Simulado sea capaz de identificar configuraciones de *rendezvous* que reduzcan el valor de $T_{\text{máx}}$ y, por tanto, mejoren la sincronización entre los dos robots. Dado que el problema combina posiciones, orientaciones y factores de velocidad, el algoritmo debería explorar de forma eficiente regiones del espacio de búsqueda que no serían accesibles mediante métodos deterministas o gradientes locales.

Además, se busca ajustar automáticamente los factores de velocidad para compensar las diferencias cinemáticas entre robots y el eje lineal.

Se considerará que el Temple Simulado constituye una estrategia válida para la optimización del *rendezvous* cooperativo si logra reducir $T_{\text{máx}}$ de forma estable y reproducible respecto del punto de partida, si muestra estabilidad de los valores finales en las últimas iteraciones, es decir, convergencia, y si el diagrama de dispersión muestra una tendencia alineada con el objetivo del problema.

Revisión y desarrollo de la propuesta técnica de búsqueda local

En esta sección se describen las correcciones introducidas a partir de la revisión del profesorado sobre la propuesta inicial de búsqueda local, se resuelve el problema y se exponen los resultados obtenidos.

Tras incorporar dichas modificaciones se observó que el algoritmo de optimización comenzaba a mostrar un nivel significativo de ruido en la evaluación del coste. Este ruido estaba asociado a la elevada dimensionalidad del espacio de búsqueda que introducía soluciones no factibles desde el inicio.

Para mitigar este efecto, se ha realizado un análisis específico orientado a reducir la dimensionalidad efectiva del problema y filtrar las soluciones no válidas antes de la optimización. En concreto, se ha resuelto el problema diseñando una arquitectura de dos niveles.

Revisión del profesorado

En la revisión de la propuesta inicial, el profesorado señaló dos aspectos que debían corregirse en el diseño del Temple Simulado. En primer lugar, se indicó que la temperatura inicial T_0 no debía fijarse de forma arbitraria, sino sintonizarse en función de la escala real de los incrementos de coste ΔJ observados en las primeras iteraciones. El objetivo es garantizar una tasa de aceptación inicial elevada (del orden del 80–90 %) para empeoramientos, evitando que el algoritmo se comporte como una búsqueda ávida (Hill Climbing) si T_0 resulta demasiado baja para la magnitud de la función de mérito.

En segundo lugar, se advirtió que el uso de una única desviación típica $\sigma = 0,1$ para todas las variables no era adecuado, ya que posiciones, orientaciones y factores de velocidad operan en escalas muy distintas. Esto puede provocar mutaciones desbalanceadas: algunas variables se hiper-mutaban mientras otras permanecían prácticamente estáticas. Por ello, se recomendó definir desviaciones típicas diferenciadas ($\sigma_p, \sigma_q, \sigma_s$) adaptadas a la naturaleza y rango de cada tipo de variable.

Estas observaciones se han incorporado en la actualización de la propuesta mediante un rediseño en dos niveles. En el nivel 1 se filtran puntos cinemáticamente válidos, reduciendo la dimensionalidad efectiva del problema y eliminando gran parte del ruido en la evaluación. En el nivel 2 se emplea un Temple Simulado con una temperatura inicial T_0 ajustada a la magnitud real de ΔJ , logrando así un comportamiento más estable, eficiente y coherente con la teoría del método.

Diseño de la búsqueda de la solución

Con la propuesta inicial se podían generar vecinos que estuvieran fuera del espacio de trabajo de los robots, por lo que desde el inicio eran puntos no factibles. Su perturbación para la generación de nuevos vecinos no incluía ningún mecanismo que pudiera acerca el nuevo punto al espacio de trabajo. Por tanto, el algoritmo tenía una alta probabilidad de ser incapaz de explorar

correctamente soluciones factibles.

Además, la naturaleza del problema implica que un mismo punto en el espacio tenga comportamientos diferentes según los perfiles de velocidad y la posición del raíl aplicada. Consecuentemente, los perfiles de velocidad y la posición del raíl tienen que ser optimizados para cada punto factible, y después, optimizar esas soluciones completas, es decir, quedarse con la mejor.

Dado este análisis, se ha propuesto una arquitectura basada en dos niveles. En el primer nivel se generan los vecinos y la información que será introducida al algoritmo de Temple Simulado. Estos vecinos se generan dentro de la intersección del espacio de trabajo de ambos robots para cuando el raíl está no se ha desplazado, puesto que en esta posición la intersección es máxima. Asimismo, estos puntos son cinemáticamente factibles, se han fijado las orientaciones acorde a la Figura 2 y se obtiene el rango de posiciones del raíl para que el robot B pueda alcanzar la posición generada.

En el segundo nivel, el algoritmo de Temple Simulado optimiza los perfiles de velocidad y la posición del raíl para la posición factible generada, generando vecinos en las variables a optimizar. En el caso del raíl, su generación de vecinos está acotada al rango obtenido en el nivel anterior, puesto que garantiza que sea alcanzable.

Tras resolver exitosamente el problema con este enfoque, se ha descubierto la posibilidad de aplicar otra metodología: establecer un mecanismo de evaluación que determine cómo de lejos se encuentra un punto del espacio de trabajo factible para que el algoritmo de Temple Simulado sea capaz de converger hacia puntos factibles. No obstante, como ya se había resuelto el problema y dado que se trata de un proyecto académico de una asignatura de 6 créditos ECTS, este nuevo enfoque se plantea como línea futura.

Diseño del algoritmo

Representación de las soluciones

Cada solución candidata se representa como un vector continuo:

$$\mathbf{x} = (\mathbf{p}_A, \mathbf{p}_B, \mathbf{q}_A, \mathbf{q}_B, s_A, s_B, s_{rail}) \in \mathbb{R}^{18}, \quad (1)$$

donde $\mathbf{p}_A \in \mathbb{R}^3, \mathbf{p}_B \in \mathbb{R}^4$ son las posiciones objetivo de cada robot en el marco del mundo, incluyendo la posición del raíl optimizada para el robot B, $\mathbf{q}_A, \mathbf{q}_B \in S^3$ son cuaterniones unitarios que representan orientaciones de cada efector y $s_A, s_B, s_{rail} \in (0, 1]$ son los factores de velocidad de cada robot y del motor lineal de desplazamiento.

Con el objetivo de simplificar el problema, el marco del mundo coincide con el marco del robot A, la base del robot B se encuentra desplaza $(0,75 + \text{posición del raíl})$ metros en el eje X , donde el raíl puede tomar cualquier valor entre 0,0 y 0,70, y las orientaciones son fijas en $\mathbf{q}_A = (0,70710678, 0,70710678, 0,0, 0,0)$ y $\mathbf{q}_B = (0,0, -0,38268343, 0,0, 0,92387953)$ (cumpliendo con la Figura 2).

Para facilitar la interpretación de dichas orientaciones, se presentan a continuación sus valores en *RPY* (*Roll, Pitch, Yaw*). En el caso del robot A ($\mathbf{q}_A$), la orientación deseada corresponde a los ángulos en radianes $(\pi, \pi/2, 0)$, lo que equivale a una rotación de 180° alrededor del eje x seguida

de una rotación de 90° alrededor del eje y . Para el robot B ($\mathbf{q}_B$), la orientación especificada es $(0, -\pi/2, 0)$, es decir, una rotación de -90° alrededor del eje y .

Operadores de vecindad

En primer lugar, se presenta la generación de vecinos para el nivel 1 de la arquitectura planteada. En este nivel el operador de vecindad ya no se basa en perturbaciones gaussianas. En su lugar, se ha diseñado un generador de vecinos específico para la naturaleza geométrica y cinemática del problema, cuyo objetivo es producir únicamente puntos factibles para ambos robots antes de iniciar la optimización temporal del nivel 2.

El proceso consta de tres etapas principales:

1. **Muestreo geométrico en el espacio de trabajo.** Se genera un punto candidato p_A mediante muestreo uniforme dentro de la intersección de los cilindros de alcance de los dos robots. A partir de él se define automáticamente el punto correspondiente del robot B como $p_B = p_A + \text{offset}$, garantizando la distancia relativa requerida entre ambos efectores, es decir, cumpliendo las restricciones definidas sobre la separación final en el mundo y la posición de enfrentamiento. Se ha establecido el *offset* a 0,2 metros en el eje x .

Los cilindros de alcance de los dos robots viene dado por el fabricante: un radio de 0,762 metros y una altura de 1,029 metros.

En la Figura 4 se observa visualmente la generación de estos puntos factibles.

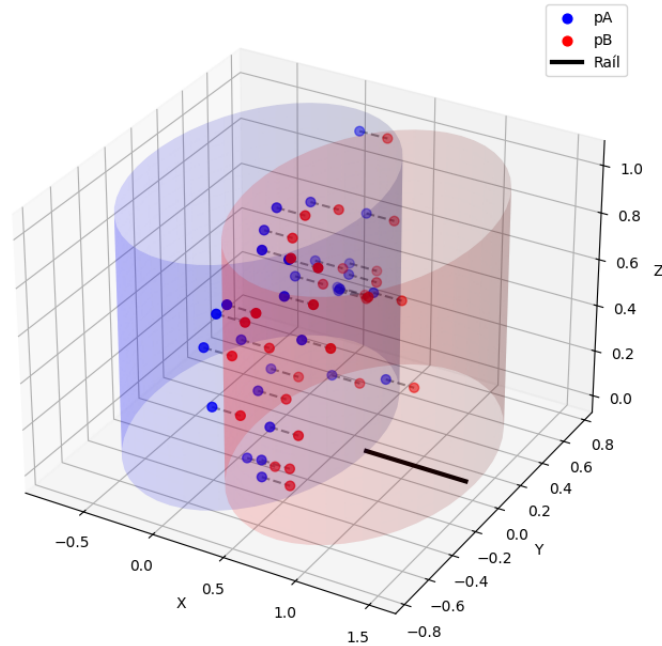


Figura 4: Visualización 3D de la generación de puntos espaciales factibles.

2. **Comprobación de factibilidad del raíl.** Para el punto p_B se calcula el intervalo de posiciones del raíl $[r_{\text{mín}}, r_{\text{máx}}]$ que permiten alcanzarlo. Si dicho intervalo es vacío, el punto se descarta. En caso contrario, se selecciona un valor intermedio $p_{\text{Raíl}}$ para evaluar la cinemática inversa.

3. **Filtrado cinemático y manipulabilidad.** Se resuelven las cinemáticas inversas de ambos robots para las orientaciones deseadas y se comprueba que las soluciones articulares obtenidas presentan una manipulabilidad superior a un umbral mínimo. Solo si se cumplen ambas condiciones, el punto se acepta como vecino válido. Este filtrado corresponde con la restricción de singularidades.

Del trabajo realizado en la asignatura de Robótica Industrial del Máster se conoce que los valores típicos de manipulabilidad en el espacio de trabajo de ambos robots presentan valores superiores a 0,05, mientras que las configuraciones cercanas a singularidades caen por debajo de dicho rango. Por este motivo se ha fijado un umbral conservador en 0,08.

En segundo lugar, el operador de vecindad del nivel 2 sí se basa en perturbaciones gaussianas, pero aplicadas únicamente a las variables temporales. Estas variables operan en rangos similares, por lo que ambas perturbaciones sí pueden utilizar la misma desviación típica, $\sigma = 0,05$, que permite explorar variaciones apreciables en el tiempo de ejecución sin producir saltos excesivos que desestabilicen el Temple Simulado.

- **Factores de velocidad.** Cada velocidad escalar se perturba mediante

$$s'_r = s_r + \mathcal{N}(0, \sigma_s^2), \quad r \in \{A, B, \text{Rail}\},$$

seguida de un truncamiento para garantizar la factibilidad física:

$$s'_r \leftarrow \min(1, \max(0, s'_r)).$$

- **Posición del raíl.** La posición final del raíl se perturba mediante

$$p'_{\text{Rail}} = p_{\text{Rail}} + \mathcal{N}(0, \sigma_{\text{rail}}^2),$$

y posteriormente se proyecta al intervalo factible calculado en el nivel 1:

$$p'_{\text{Rail}} \leftarrow \min(r_{\text{máx}}, \max(r_{\text{mín}}, p'_{\text{Rail}})).$$

Esta operación garantiza que el robot B pueda alcanzar el punto p_B tras la mutación.

Gestión de restricciones

Como se ha venido explicando, la arquitectura implementada junto con los operadores de vecindad cumplen la mayoría de las restricciones gracias a la incorporación del conocimiento del dominio. De esta manera, la única restricción a verificar por el algoritmo del Temple Simulado será la sincronización temporal:

$$|t_A(\mathbf{p}_A, \mathbf{q}_A, \mathbf{s}_A) - t_B(\mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_B, \mathbf{s}_{\text{rail}})| \leq \Delta t_{\text{máx}}.$$

Por tanto, la evaluación de cada solución se basa en esta diferencia temporal: cuanto más próximos estén ambos tiempos, mejor será la calidad de la solución. Se ha modificado la función de penalización correspondiente para cumplir con este planteamiento:

$$P_{\text{sync}}(\Delta t) = \begin{cases} \left(\frac{\Delta t}{\Delta t_{\text{máx}}}\right)^2, & \text{si } \Delta t \leq \Delta t_{\text{máx}}, \\ 1 + \left(\frac{\Delta t - \Delta t_{\text{máx}}}{\Delta t_{\text{máx}}}\right)^2, & \text{si } \Delta t > \Delta t_{\text{máx}}. \end{cases}$$

Es importante destacar que con esta modificación la restricción de sincronización no se aplica como un filtro duro que descarte soluciones que excedan el umbral $\Delta t_{\text{máx}}$. En su lugar, la restricción se incorpora mediante una *penalización continua* que permite al Temple Simulado seguir explorando el espacio de soluciones incluso cuando la diferencia temporal es superior al límite deseado. De este modo se evita la aparición de zonas ciegas en la función de mérito y se mantiene un gradiente útil para guiar la búsqueda.

De este modo, el Temple Simulado explora libremente el espacio de búsqueda, mientras que la función de evaluación guía la convergencia hacia soluciones que cumplen las restricciones, en este caso, la sincronización temporal.

Configuración de parámetros

Conociendo el modelo del tiempo de ejecución de cada robot utilizado, descrito en la propuesta de esta memoria, los datos geométricos del problema, el alcance máximo de los robots y el recorrido máximo del raíl, es posible acotar de forma aproximada los tiempos de ejecución:

$$t_r \in [t_{\text{mín}}, t_{\text{máx}}] \approx [0,5 \text{ s}, 2,5 \text{ s}].$$

En consecuencia, la desincronización máxima teórica entre ambos robots se sitúa en el orden de

$$\Delta t_{\text{máx}}^{\text{teo}} \approx t_{\text{máx}} - t_{\text{mín}} \approx 2 \text{ s}.$$

Sin embargo, el operador de vecindad del nivel 2 no genera saltos extremos, sino pequeñas variaciones en los factores de velocidad y en la posición del raíl ($\sigma_s = 0,05$, $\sigma_{\text{raíl}} = 0,05$). Esto implica que la diferencia de tiempos entre soluciones vecinas se sitúa típicamente en el orden de unas pocas décimas de segundo, lo que se traduce en una variación de coste esperada:

$$\Delta J_{\text{esp}} \sim P_{\text{sync}}(\Delta t) - P_{\text{sync}}(\Delta t') \quad \Rightarrow \quad \Delta J_{\text{esp}} \in [0,1, 0,3].$$

Esta estimación permite sintonizar la temperatura inicial del Temple Simulado sin necesidad de recurrir a ejecuciones preliminares ni a ajustes empíricos. El objetivo de la temperatura inicial es garantizar una probabilidad de aceptación elevada (80–90 %) para empeoramientos típicos de magnitud ΔJ_{esp} . Utilizando la relación clásica del Temple Simulado,

$$T_0 = -\frac{\Delta J_{\text{esp}}}{\ln(p_0)}, \quad p_0 \in [0,8, 0,9],$$

y tomando como referencia un valor medio $\Delta J_{\text{esp}} \approx 0,2$, se obtiene:

$$T_0 \approx -\frac{0,2}{\ln(0,85)} \approx 0,96.$$

Por simplicidad se adopta finalmente

$$T_0 = 1,$$

valor coherente con la escala temporal del problema y que evita que el algoritmo degenera en una búsqueda puramente ávida en las primeras iteraciones.

Por otra parte, el enfriamiento empleado es de tipo geométrico, $T_{k+1} = \alpha T_k$, por ser el más habitual en problemas continuos y por su buena relación entre simplicidad y estabilidad. La elección del factor de enfriamiento α se ha realizado en función del número total de iteraciones disponibles y de la escala de la función de mérito.

Se define un máximo de $\text{max_iters} = 300$ iteraciones, por lo que el factor de enfriamiento debe garantizar que la temperatura se mantenga suficientemente alta durante las primeras fases (para permitir exploración) y que descienda de forma progresiva hacia valores cercanos a cero en las últimas iteraciones (para favorecer la explotación). Para un enfriamiento geométrico, la temperatura tras N iteraciones es:

$$T_N = T_0 \alpha^N.$$

Imponiendo que la temperatura final sea aproximadamente dos órdenes de magnitud menor que la inicial, es decir, $T_N \approx 10^{-2}T_0$, se obtiene:

$$\alpha^N \approx 10^{-2} \Rightarrow \alpha \approx 10^{-2/N}.$$

Sustituyendo $N = 300$ se obtiene:

$$\alpha \approx 10^{-2/300} \approx 0,98.$$

Por tanto, el valor $\alpha = 0,98$ no es arbitrario, sino que garantiza un descenso suave de la temperatura a lo largo de las 300 iteraciones disponibles, manteniendo capacidad exploratoria en las primeras fases y favoreciendo la convergencia en las finales.

El proceso de optimización temporal utiliza un máximo de $\text{max_iters} = 300$ iteraciones. Este valor sustituye al esquema clásico de dos bucles del Temple Simulado (con K temperaturas y L vecinos por temperatura), debido a la propia naturaleza de la arquitectura en dos niveles.

En un Temple Simulado convencional, el esquema $K \times L$ es necesario para compensar la presencia de un espacio de búsqueda ruidoso, con restricciones complejas y con una elevada probabilidad de generar soluciones no factibles. Sin embargo, en la arquitectura propuesta, el nivel 1 garantiza que todas las soluciones que llegan al nivel 2 son geométrica y cinemáticamente factibles, están libres de singularidades y cumplen las restricciones del raíl. Como consecuencia, el espacio de búsqueda del nivel 2 es continuo, de baja dimensionalidad y completamente factible, por lo que no requiere un proceso de exploración tan intensivo.

Además, el operador de vecindad del nivel 2 produce variaciones pequeñas y controladas en los parámetros temporales, lo que reduce drásticamente el ruido en la función de mérito. En este contexto, el uso de un único bucle con un número total de iteraciones fijo resulta más eficiente y permite un control más directo sobre el coste computacional.

En la práctica, 300 iteraciones proporcionan un equilibrio adecuado entre exploración y explotación, permitiendo alcanzar convergencia estable sin incrementar de forma excesiva el tiempo de cómputo. En otras palabras, se probarán 300 combinaciones de soluciones por cada punto del nivel 1, siendo un número adecuado dado el amplio espacio de búsqueda combinatorio definido por las cuatro variables continuas a optimizar en el Temple Simulado.

Diseño experimental

El plan de experimentación constará de la optimización de 30 puntos espaciales diferentes.

Las soluciones se evaluarán mediante tres métricas: calidad, tiempo de ejecución y número de evaluaciones. La calidad registra el mejor y peor valor final de $T_{\text{máx}}$, la media, la desviación típica y el intervalo de confianza. El tiempo de ejecución mide el tiempo total de *CPU* y se considera con la calidad de la solución lograda. Esta consideración se realizará a través de un diagrama de

dispersión, como el de la Figura 3, y un valor de eficiencia calculado: $E = \frac{T_{max}}{\text{tiempo}}$. Por último, el número de evaluaciones anota la iteración en la que se alcanzó el mejor valor.

Resultados

Se han realizado modificaciones sobre la representación de resultados propuesta para evitar la redundancia de información. La Tabla 1 y la Figura 3 contenían información duplicada y además, la tabla resultaría difícil de leer por su gran volumen de datos. Por ello, se ha decidido suprimir la tabla.

Además, con el objetivo de analizar la estabilidad del método, evaluar la variabilidad entre ejecuciones y estudiar el comportamiento dinámico del algoritmo durante el proceso de optimización se añaden dos diagramas de cajas y la curva media de convergencia. Un diagrama de caja representa el valor de $T_{m\acute{a}x}$ y otro los diferentes valores de la coordenada x de los puntos A y B, puesto que las coordenadas y y z están fijadas durante el proceso de optimización.

Por otro lado, con los cambios realizados ya no existen soluciones infactibles, por lo que no es necesario registrar el porcentaje de soluciones factibles obtenidas por el algoritmo.

El mejor resultado obtenido viene definido por $T_{m\acute{a}x} = 0,96 \text{ s}$ con el vector número 7 (recordar que las orientaciones están previamente fijadas):

$$(p_A, p_B, q_A, q_B, s_A, s_B, s_{rail}) = ((0,35, 0,12, 0,15), (0,55, 0,12, 0,15, p_{rail} = 0,5), q_A, q_B, 0,41, 1, 0,84)$$

Los resultados de la Tabla 3 muestran que el algoritmo presenta un comportamiento estable y consistente a lo largo de las 30 ejecuciones realizadas. El valor medio de $T_{m\acute{a}x}$ es de 1.67 s, con una desviación típica moderada (0.50 s), lo que indica que la variabilidad entre ejecuciones es relativamente baja. El mejor caso alcanza un tiempo de 0.96 s, mientras que el peor se sitúa en 2.88 s, lo que define el rango completo de rendimiento del método. El intervalo de confianza al 95 % [1,49, 1,88] s confirma que, con alta probabilidad, el algoritmo converge de forma consistente en torno a valores próximos a la media.

Por otro lado, el tiempo medio de ejecución del proceso completo es de 0.83 s, lo que demuestra que el método es computacionalmente eficiente y adecuado para escenarios donde se requiere optimización rápida.

De forma complementaria, el diagrama de cajas de la Figura 5 muestra que la mayor parte de las soluciones de $T_{m\acute{a}x}$ se concentran en un intervalo relativamente estrecho. El rango intercuartílico se sitúa aproximadamente entre 1.25 s y 1.90 s, lo que indica que el 50 % central de las ejecuciones produce soluciones de calidad similar. El bigote inferior desciende hasta aproximadamente 1.00 s, mientras que el superior alcanza valores cercanos a 2.90 s, lo que refleja la presencia de ejecuciones más extremas, tanto muy buenas como menos favorables. No obstante, la ausencia de valores atípicos sugiere que estas variaciones se encuentran dentro del comportamiento esperado del método.

Métrica	Valor
Media de $T_{\text{máx}}$	1.67 s
Desviación típica	0.5
Mejor valor (mínimo)	0.96 s
Peor valor (máximo)	2.88 s
Intervalo de confianza 95 %	[1.49, 1.88] s
Tiempo medio de ejecución (s)	0.83 s

Tabla 3: Métricas agregadas calculadas sobre las 30 ejecuciones.

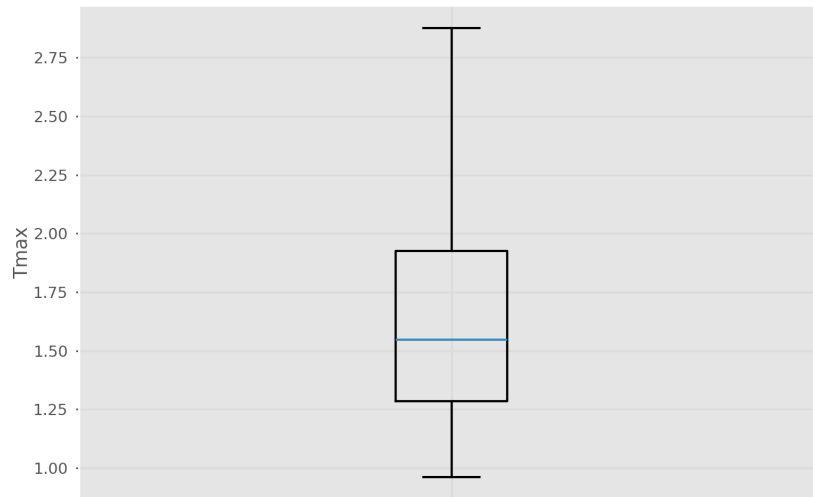


Figura 5: Diagrama de caja de la solución temporal.

También resulta interesante evaluar la variabilidad en los puntos espaciales explorados. En la Figura 6 se presenta el diagrama de caja para la coordenada x de los puntos A y B. La mayoría de las soluciones se agrupan en el intervalo $[0,2, 0,4]$ metros y $[0,4, 0,6]$ metros, lo que induce a pensar que esas zonas son las regiones más estables, alcanzables o cinemáticamente factibles para los robots. En la Figura 7 se visualizan en tres dimensiones los 30 puntos explorados.

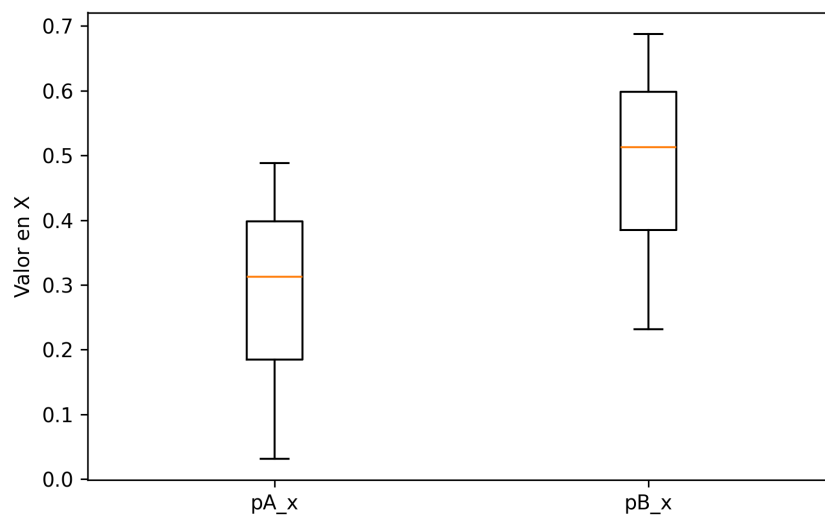


Figura 6: Diagrama de caja del punto espacial.

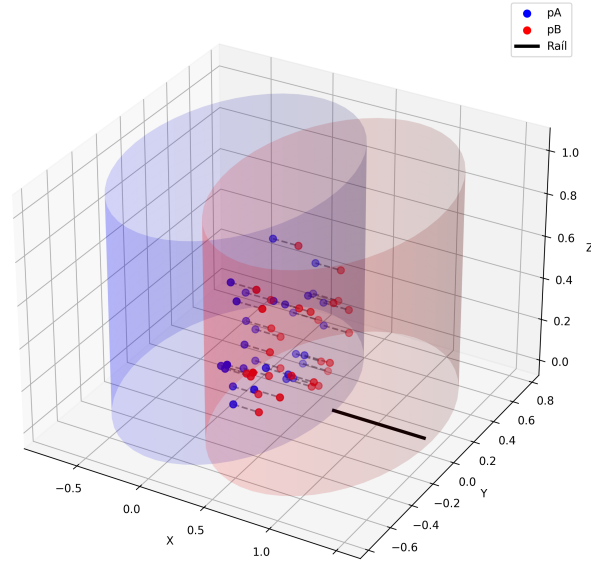


Figura 7: Visualización de los puntos espaciales factibles optimizados.

En lo relativo a la eficiencia, la Figura 8 muestra que la métrica propuesta no aporta nada útil a la evaluación, ya que las soluciones con mayor tiempo de desplazamiento de los robots calculada en el menor tiempo de computación tiene una mayor eficiencia. Esta relación no aporta ningún significado. La única información de interés que se obtiene es que el tiempo máximo de ejecución es de 3 segundos por punto espacial. Consecuentemente, resulta necesario definir otra métrica. En este caso, se ha optado por la curva de convergencia del método.

Esta curva se presenta en la Figura 13, donde se observa que el algoritmo ha convergido a las 80 iteraciones de las 300 que se realizan, ya que la penalización es prácticamente nula. Es importante justificar el uso de J para graficar esta curva, ya que aunque la función de mérito utilizada en el nivel 2 del Temple Simulado se reduce a la penalización de sincronización $J = P_{\text{sync}}(\Delta t)$, $\Delta t = |t_A - t_B|$, esta magnitud actúa como un mecanismo indirecto de minimización del tiempo dentro del punto espacial seleccionado en el nivel 1, que sería la función objetivo.

Para que la penalización sea mínima, el algoritmo debe ajustar los factores de velocidad y la posición del raíl de modo que ambos robots alcancen el punto de cooperación en el mismo instante. Esta coincidencia temporal solo puede lograrse si los tiempos individuales se igualan en el menor valor posible compatible con la geometría del punto. En consecuencia, aunque el nivel 2 no modifica la posición espacial (que determina el tiempo mínimo alcanzable), sí minimiza el tiempo de ejecución dentro de ese punto al forzar la sincronización.

Por este motivo, las curvas de convergencia representan la evolución de la función de mérito J , que es la magnitud que realmente guía el proceso de optimización.

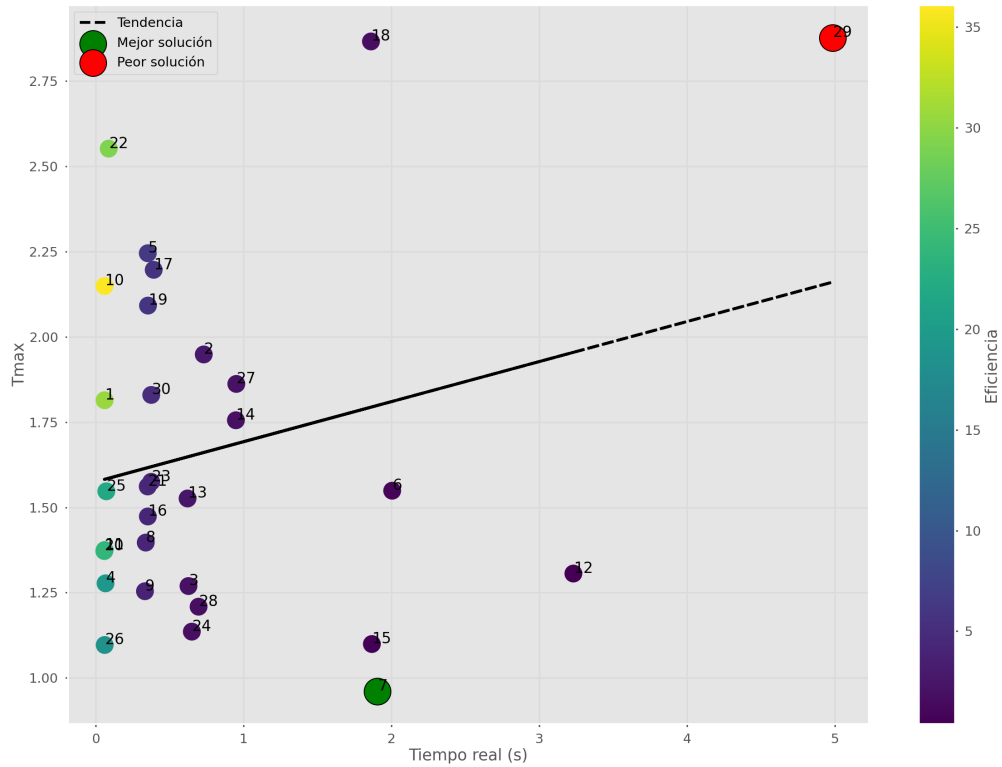


Figura 8: Diagrama de dispersión propuesto para la evaluación de la eficiencia de las soluciones. Cada círculo representa una solución cuyo número adjunto corresponde con el número de ejecución, el color de las soluciones se refiere a la eficiencia de las mismas, los círculos con borde negro representan las peor y mejor solución, en color rojo y verde, respectivamente, y la recta punteada la línea de tendencia.

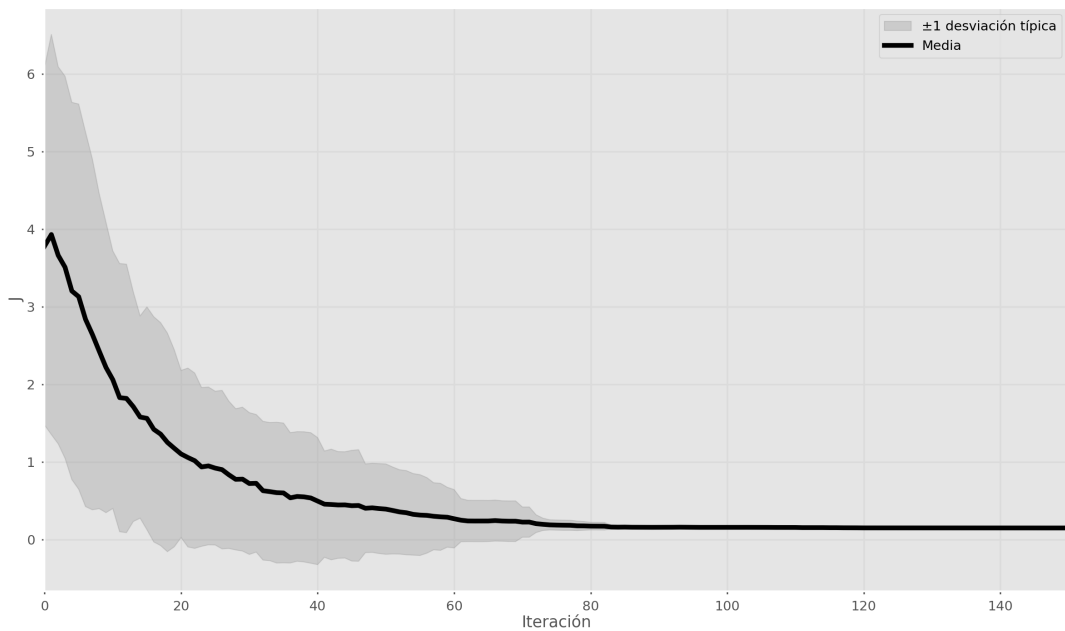


Figura 9: Curva media de convergencia del método desarrollado, donde se aprecia la disminución de la penalización J .

Conclusiones

Durante la evaluación de los resultados se ha observado que el algoritmo tiende a converger de forma prematura, lo que sugiere que no está explorando completamente el espacio de soluciones disponible. Sin embargo, las soluciones proporcionadas son muy lógicas, salvo por los perfiles de velocidad cuando estos reciben el máximo valor posible (1).

Resulta evidente que la mejor solución, es decir, aquella con menor tiempo de desplazamiento, se obtiene con la máxima velocidad de los robots. Esta situación está introduciendo un sesgo al algoritmo para explorar únicamente perfiles elevados de velocidad, reduciendo el espacio de búsqueda, provocando una convergencia prematura y priorizando la minimización del tiempo a costa de ignorar criterios de calidad del movimiento que serían relevantes en un escenario real.

Así todo, no todos los perfiles de velocidad optimizados son elevados, puesto que en la mejor solución obtenida la escala de velocidad del robot A es de 0,41. Consecuentemente, demuestra que a pesar de que el algoritmo tenga un sesgo, es capaz de optimizar parcialmente el punto de encuentro de ambos robots.

La eliminación de este sesgo se plantea como línea futura, mediante la introducción de alguna restricción sobre aceleración, suavidad o calidad de movimiento.

Además, el método implementado es computacionalmente eficiente puesto que los tiempos de ejecución son muy bajos, pudiendo ser utilizado en aplicaciones de planificación de trayectorias de robots.

Por último, otro trabajo futuro que se plantea es la resolución del problema con la metodología mencionada previamente: establecer un mecanismo de evaluación que determine cómo de lejos se encuentra un punto del espacio de trabajo factible para que el algoritmo de Temple Simulado sea capaz de converger hacia puntos factibles.

Propuesta técnica de búsqueda evolutiva

Algoritmo elegido

Se ha seleccionado la técnica de Evolución Diferencial (*DE*). Esta elección se justifica por la naturaleza del problema definido en los Temas 1 y 2, que opera en un espacio de búsqueda continuo con variables reales. Se trabajará sobre el planteamiento derivado de la revisión del Tema 2. Cabe destacar que al principio, dadas las restricciones del problema, especialmente la relación espacial de los puntos de encuentro, se planteó el algoritmo de Nubes de Partículas (*PSO*). Sin embargo, tras la tutorización del equipo docente y a la vista de los resultados obtenidos en el Tema 2, se determinó que la *DE* tenderá a mantener una mayor diversidad poblacional y ser menos propensa a la convergencia prematura.

Diseño del algoritmo

Representación de las soluciones

Cada solución candidata se representa como el vector continuo de la Ecuación 1.

Planteamiento del problema

Se continúa con el planteamiento mono-objetivo penalizado que se ha desarrollado en la revisión del Tema 2, adoptando una arquitectura de dos niveles para reducir la dimensionalidad y filtrar soluciones cinemáticamente inválidas antes de la optimización. Al igual que con la búsqueda local, el algoritmo evolutivo se aplica al Nivel 2, donde el vector de decisión se centra en las variables continuas que regulan la ejecución temporal y la posición final del raíl:

$$\mathbf{x} = (s_A, s_B, s_{rail}, p_{rail}) \in \mathbb{R}^4 \quad (2)$$

Operadores evolutivos

Se utilizará la variante estándar **DE/rand/1/bin**:

- **Inicialización:** población generada aleatoriamente mediante una distribución uniforme en el espacio de búsqueda $(0, 1]$ para garantizar diversidad inicial. El rango de las variables de velocidad es $(0, 1]$, por lo que se proyecta el rango real de la posición del raíl a ese mismo intervalo para facilitar los mecanismos del algoritmo.
- **Perturbación:** se genera un vector mutante mediante la diferencia escalada de dos individuos aleatorios sumada a un tercero: $v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$.
- **Cruce:** cruce binomial (uniforme) que combina el vector actual con el mutante basado en una probabilidad CR .
- **Selección:** selección elitista 1 a 1, donde el hijo sustituye al padre solo si mejora su valor de aptitud o factibilidad.
- **Reparación:** tras el cruce, se aplica una función de reparación para proyectar la posición del raíl obtenida a sus límites físicos permitidos.

Método de evaluación

Se va a emplear el mismo tratamiento de restricciones que en el Tema 2.

Configuración de parámetros

Siguiendo las indicaciones del equipo docente y las recomendaciones técnicas para optimización en espacios continuos, se propone la siguiente configuración:

- **Tamaño de la población (NP) de 60:** se justifica bajo la regla heurística de emplear entre $10d$ y $20d$, siendo $d = 4$ la dimensionalidad del DE .
- **Factor de escala (F) de 0.75:** situado en el rango recomendado de $[0,5, 0,8]$, permite que las perturbaciones diferenciales basadas en la diversidad de la población sean lo suficientemente grandes para saltar óptimos locales.
- **Probabilidad de cruce (CR) de 0.90:** se selecciona un valor alto debido al fuerte acoplamiento entre las velocidades de los robots y la posición del raíl. Esto asegura que el vector hijo herede la mayor parte de la información del vector mutante perturbado, evitando el estancamiento en regiones subóptimas.
- **Máximo de generaciones (150):** este valor se ha fijado para igualar el coste computacional del Tema 2. Dado que en el Temple Simulado se realizaron 300 iteraciones para cada uno de los 30 puntos espaciales (9,000 evaluaciones totales), establecer 150 generaciones con una población de 60 individuos genera exactamente el mismo número de soluciones ($60 \cdot 150 = 9,000$).

Además se proponen tres grupos más de parámetros para realizar un análisis de sensibilidad de *Kruskal-Wallis*: una configuración exploratoria ($F = 0,4$, $CR = 0,5$), una intermedia ($F = 0,6$, $CR = 0,7$) y una con perturbaciones agresivas ($F = 0,9$, $CR = 0,9$).

Diseño experimental

El plan de experimentación consistirá en 30 ejecuciones independientes por cada grupo de parámetros a evaluar mediante el test estadístico de *Kruskal-Wallis*. Finalmente, se seleccionará la configuración más adecuada y sus resultados se contrastarán con los obtenidos en el tema anterior, utilizando para ello los mismos diagramas, esquemas y tablas.

Desarrollo de la propuesta técnica de búsqueda evolutiva

En esta sección se resuelve el problema mediante la técnica evolutiva elegida en la propuesta del apartado anterior y se exponen los resultados obtenidos.

Se ha realizado un análisis de sensibilidad de *Kruskal-Wallis* con el objetivo de estudiar la sensibilidad de los parámetros (F , CR) propios del algoritmo de evolución diferencial.

Resultados principales

Los resultados de la Tabla 4 muestran que el algoritmo presenta un comportamiento estable y consistente a lo largo de las 30 ejecuciones realizadas. Además, ha obtenido mejores resultados que el algoritmo de búsqueda local (Tabla 3), valores que se incluyen en la Tabla 4 para realizar la comparativa.

El valor medio de $T_{\max}$ es de 1.31 s, con una desviación típica moderada (0.23 s), lo que indica que la variabilidad entre ejecuciones es aún más baja que en el tema anterior. Esto es un indicador de que el algoritmo ha sido más eficaz a la hora encontrar cuál es el espacio de modificaciones en el vector de soluciones para satisfacer en mayor grado a la función objetivo. El mejor caso alcanza un tiempo de 0.86 s, mientras que el peor se sitúa en 1.67 s, lo que define el rango completo de rendimiento del método. El intervalo de confianza al 95 %, [1.23, 1.39] s, confirma que, con alta probabilidad, el algoritmo converge de forma consistente en torno a valores próximos a la media.

Por otro lado, el tiempo medio de ejecución del proceso completo es de 1.98 s, lo que demuestra que el método, aunque más costoso, sigue siendo computacionalmente eficiente y adecuado para escenarios donde se requiere optimización rápida. A pesar de ese mayor coste, la mejora en los resultados lo compensa.

De forma complementaria, el diagrama de cajas de la Figura 10 muestra que la mayor parte de las soluciones de $T_{\max}$ se concentran en un intervalo relativamente estrecho. El rango intercuartílico se sitúa aproximadamente entre 1.18 s y 1.49 s, lo que indica que el 50 % central de las ejecuciones produce soluciones de calidad similar. El bigote inferior desciende hasta aproximadamente 0.85 s, mientras que el superior alcanza valores cercanos a 1.75 s, lo que refleja la presencia de ejecuciones tanto muy buenas como menos favorables. No obstante, la ausencia de valores atípicos sugiere que estas variaciones se encuentran dentro del comportamiento esperado del método.

Métrica	Valor	Valor anterior (tema 2)	Diferencia
Media de $T_{\max}$	1.31 s	1.67	-21.6 %
Desviación típica	0.23	0.50	-54.0 %
Mejor valor (mínimo)	0.86 s	0.96	-10.4 %
Peor valor (máximo)	1.67 s	2.88	-42.0 %
Intervalo de confianza 95 %	[1.23, 1.39] s	[1.49, 1.88]	-17.4 %
Tiempo medio de ejecución (s)	1.98 s	0.83	+138.6 %

Tabla 4: Métricas agregadas calculadas sobre las 30 ejecuciones de evolución diferencial.

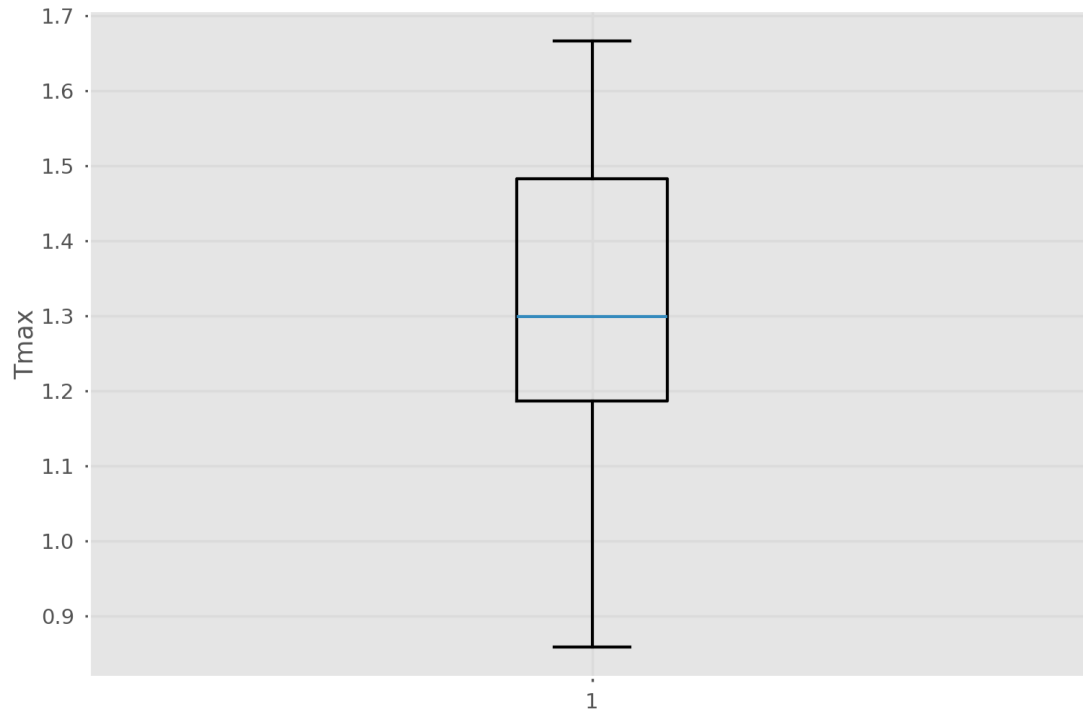


Figura 10: Diagrama de caja de la solución temporal evolutiva.

También resulta interesante evaluar la variabilidad en los puntos espaciales explorados. En la Figura 11 se presenta el diagrama de caja para la coordenada x de los puntos A y B. La mayoría de las soluciones se agrupan en el intervalo $[0,2, 0,4]$ metros y $[0,4, 0,6]$ metros, lo que induce a pensar que esas zonas son las regiones más estables, alcanzables o cinemáticamente factibles para los robots. En la Figura 12 se visualizan en tres dimensiones los 30 puntos explorados.

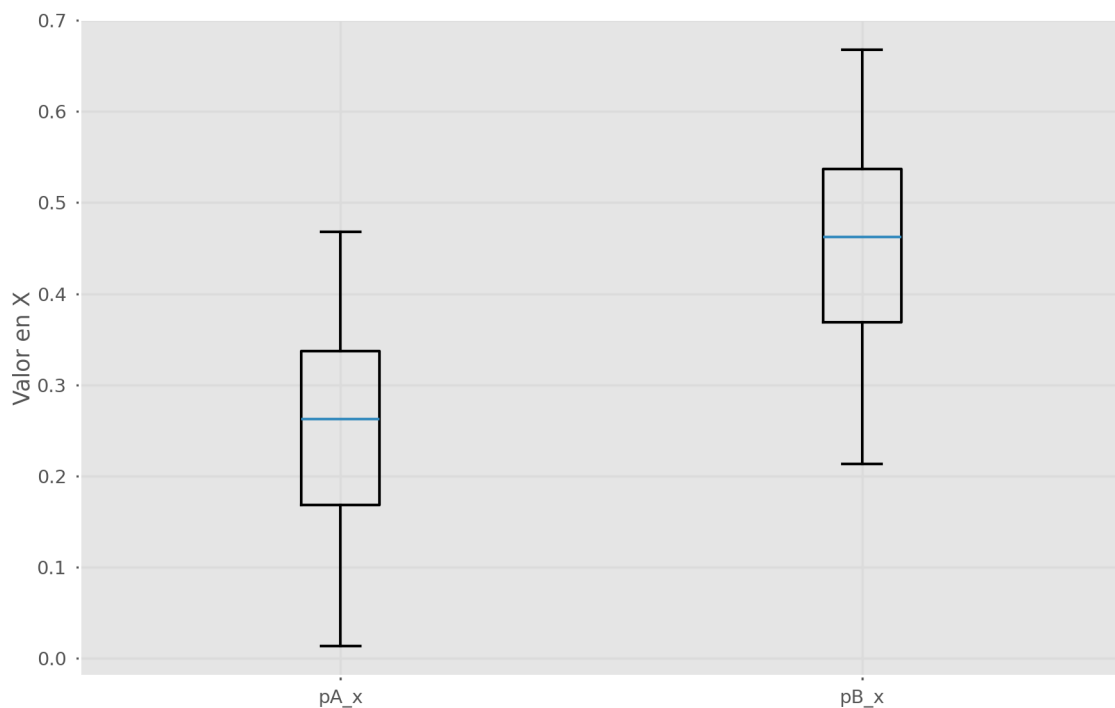


Figura 11: Diagrama de caja del punto espacial en el tema 3.

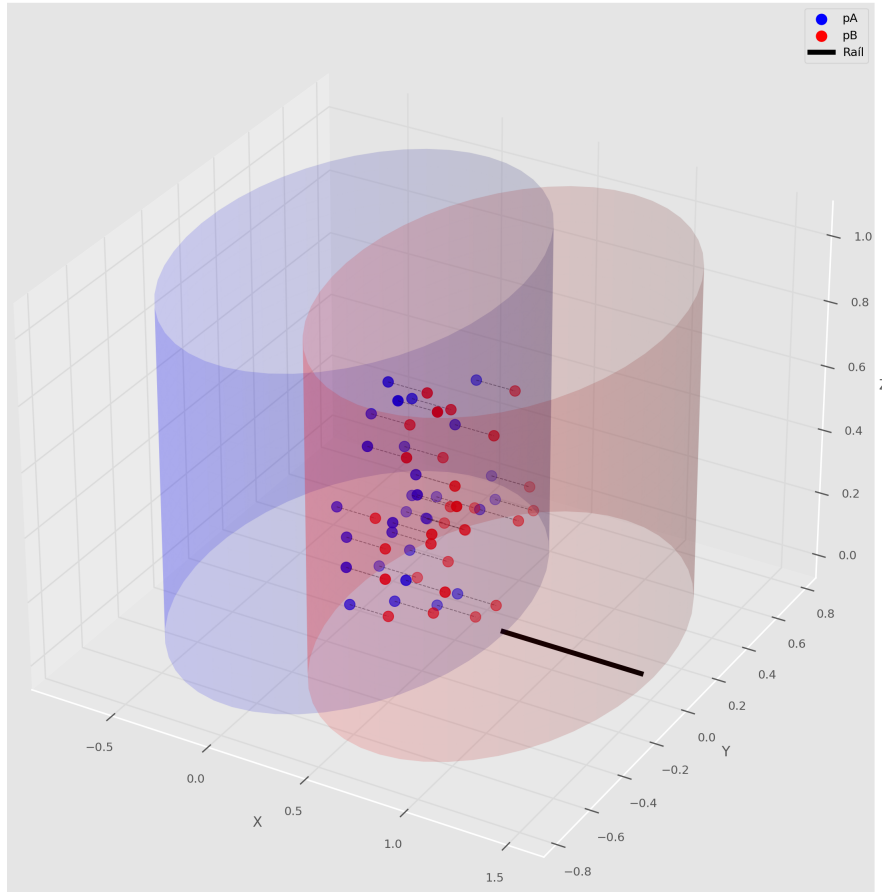


Figura 12: Visualización de los puntos espaciales factibles optimizados en el tema 3.

En la Figura 13 se presenta la curva de convergencia del método, donde se observa que el algoritmo ha convergido a las 3 generaciones de las 150 que se realizan. Este resultado, aunque llamativo, no implica necesariamente una convergencia prematura. En realidad, es coherente con la naturaleza del problema: en esta fase, el espacio efectivo del vector de decisión se reduce a un hipercubo de únicamente cuatro dimensiones (posición del raíl y tres factores de velocidad) y la evaluación propuesta resulta trivial para el algoritmo. En un dominio tan compacto, la evolución diferencial es capaz de desplazarse con gran rapidez hacia regiones prometedoras, estabilizando el valor de la función objetivo en muy pocas generaciones.

Además, los resultados obtenidos en la tabla anterior muestran valores consistentes y estables entre ejecuciones, lo que indica que el algoritmo no solo converge rápido, sino que realiza una búsqueda ágil y de calidad, identificando de forma fiable zonas factibles y de buen rendimiento. La ausencia de mejoras posteriores no se debe a estancamiento, sino a que el paisaje del problema, dominado por restricciones geométricas y cinemáticas muy estructuradas, ofrece pocas oportunidades adicionales de optimización una vez alcanzada la región óptima.

Es importante justificar el uso de J para graficar esta curva, ya que aunque la función de mérito utilizada en el nivel 2 se reduce a la penalización de sincronización $J = P_{\text{sync}}(\Delta t)$, $\Delta t = |t_A - t_B|$, esta magnitud actúa como un mecanismo indirecto de minimización del tiempo dentro del punto espacial seleccionado en el nivel 1, que sería la función objetivo.

Para que la penalización sea mínima, el algoritmo debe ajustar los factores de velocidad y la posición del raíl de modo que ambos robots alcancen el punto de cooperación en el mismo instante. Esta coincidencia temporal solo puede lograrse si los tiempos individuales se igualan en

el menor valor posible compatible con la geometría del punto. En consecuencia, aunque el nivel 2 no modifica la posición espacial (que determina el tiempo mínimo alcanzable), sí minimiza el tiempo de ejecución dentro de ese punto al forzar la sincronización.

Por este motivo, las curvas de convergencia representan la evolución de la función de mérito J , que es la magnitud que realmente guía el proceso de optimización.

Asimismo, dada la rapidez de convergencia del algoritmo, resulta interesante evaluar también la evolución del tiempo máximo obtenido a lo largo de las generaciones. En la Figura 14 se observa que el valor óptimo se obtiene habitualmente a partir de la generación 50. Estos resultados se tienen que valorar conjuntamente con la curva de convergencia de la Figura 13 para entender que a partir de la tercera generación, el algoritmo comienza a explotar la región óptima.

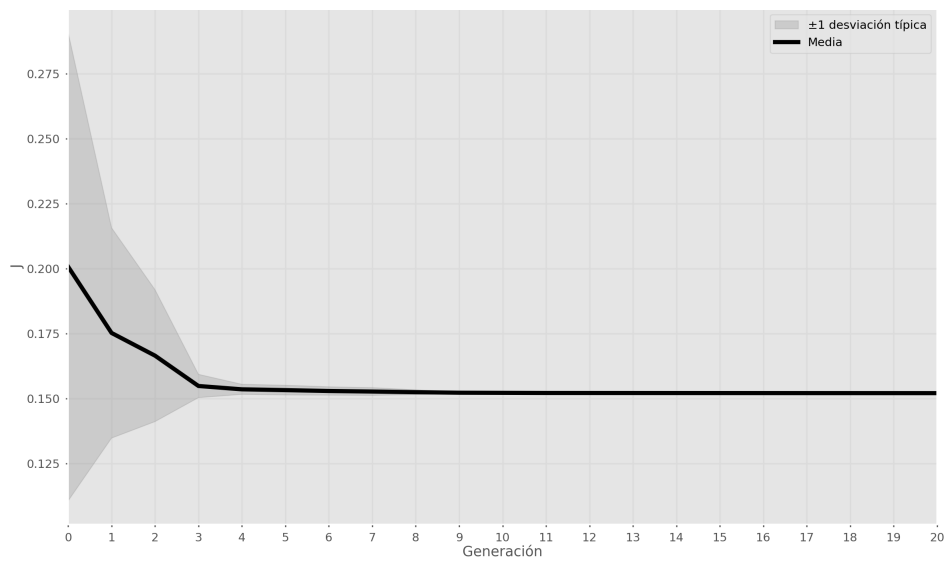


Figura 13: Curva media de convergencia de la evolución diferencial, donde se aprecia la disminución de la penalización J .

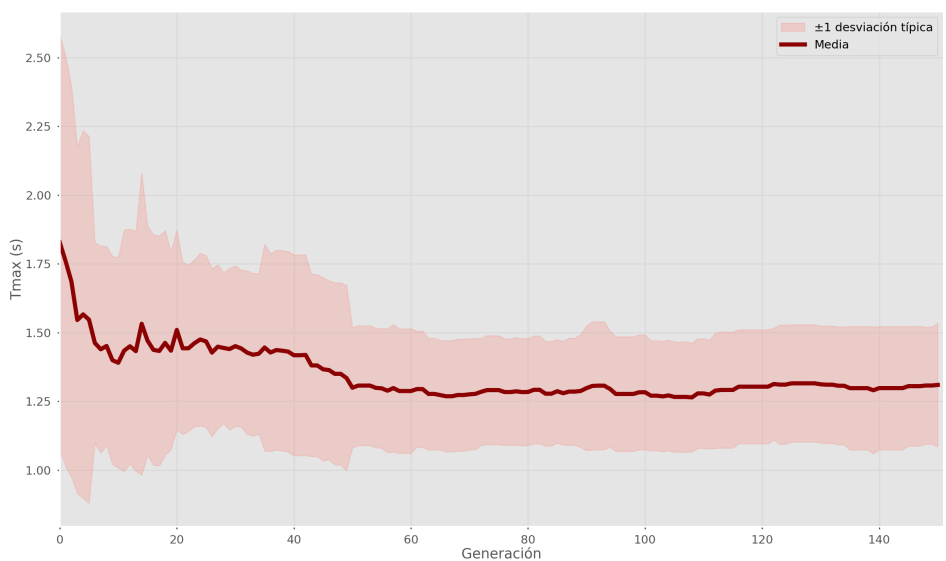


Figura 14: Evolución del tiempo máximo de desplazamiento obtenido.

Para evaluar la sensibilidad del algoritmo frente a variaciones en los parámetros, se aplicó un test no paramétrico de *Kruskal-Wallis* [5] sobre las distribuciones de $T_{\text{máx}}$ obtenidas en las cuatro configuraciones analizadas. Este test contrasta la hipótesis nula

$$H_0 : \text{todas las configuraciones generan la misma distribución de } T_{\text{máx}},$$

frente a la hipótesis alternativa

$$H_1 : \text{al menos una configuración produce una distribución distinta.}$$

El análisis se realizó mediante la función `kruskal()` de la librería *scipy.stats*. El resultado del test fue un estadístico $H = 0,27$ con un valor $p = 0,965$. Un valor de H tan reducido indica que las medianas de los grupos son prácticamente indistinguibles, mientras que un p -valor tan elevado implica que las diferencias observadas entre configuraciones son totalmente compatibles con el azar. Por tanto, no existe evidencia estadística para rechazar la hipótesis nula, lo que confirma que los parámetros evaluados no influyen de manera apreciable en el rendimiento del algoritmo.

Como se ha explicado con la convergencia del método, este comportamiento también es coherente con la naturaleza del problema ya que el espacio de búsqueda efectivo se reduce a un hiper cubo de únicamente cuatro dimensiones (posición del raíl y tres factores de velocidad). En un dominio tan reducido, las distintas configuraciones del algoritmo exploran regiones muy similares y convergen hacia soluciones prácticamente idénticas. Como consecuencia, la evolución diferencial muestra un comportamiento consistente y robusto, manteniendo valores similares de $T_{\text{máx}}$ independientemente de la configuración empleada, tal y como se recoge en la Tabla 5.

F	CR	$T_{\text{máx}} \text{ (s)}$
0,4	0,5	0,89
0,6	0,7	0,91
0,75	0,9	0,86
0,9	0,9	0,89

Tabla 5: Resultados de $T_{\text{máx}}$ para distintas configuraciones de F y CR .

Conclusiones

Se ha observado que el algoritmo de Evolución Diferencial tiende a converger rápidamente, obteniendo además mejores resultados que el Temple Simulado implementado en el tema 2. Dado que los resultados siguen siendo plenamente coherentes entre ejecuciones, se deduce que la arquitectura de descomposición del problema en dos niveles ha simplificado enormemente la búsqueda, reduciendo el espacio efectivo a un hiper cubo de baja dimensión. Esta reducción explica tanto la rapidez de convergencia como la estabilidad de las soluciones obtenidas. No obstante, incluso dentro de este espacio reducido, la propia naturaleza del algoritmo de Evolución Diferencial ha demostrado operar de forma más eficiente y con menos iteraciones que la búsqueda local del Temple Simulado, identificando con mayor rapidez la región óptima.

Además, el análisis estadístico mediante el test no paramétrico de *Kruskal-Wallis* confirma la robustez del método frente a variaciones en los parámetros. El estadístico obtenido ($H = 0,27$) junto con un valor $p = 0,965$ indica que no existen diferencias estadísticamente significativas entre las configuraciones evaluadas, lo que refuerza la conclusión de que el algoritmo mantiene

un comportamiento estable independientemente de los valores de F y CR . Este resultado es coherente con el reducido tamaño del espacio de búsqueda efectivo, que hace que todas las configuraciones exploren regiones muy similares y converjan hacia soluciones prácticamente idénticas.

Para explotar completamente el potencial del método, sería deseable que el propio algoritmo de Evolución Diferencial se encargase también del nivel 1, es decir, de la optimización de los puntos espaciales de *rendezvous*. Sin embargo, esta extensión requiere disponer de una cinemática inversa analítica para los robots utilizados. La cinemática inversa numérica de la *Robotics Toolbox* de Peter Corke resulta computacionalmente demasiado costosa para el número de evaluaciones necesarias, especialmente considerando la penalización asociada a la singularidad en la función de mérito. Se exploró el uso de la librería *ssik*, que sí ofrece soluciones analíticas para los manipuladores empleados, pero esta requiere $Python \geq 3.11$, mientras que la *Robotics Toolbox* sólo es compatible con versiones anteriores. Esta incompatibilidad hace que la integración de ambas herramientas requiera un trabajo adicional de preparación y adaptación que queda fuera del alcance de la asignatura.

Aunque se dispone de un *solver* operativo en *ROS2* basado en *Movel2* y *FastIK*, su utilización en esta aplicación concreta resulta poco práctica. La complejidad intrínseca de la arquitectura de *ROS2*, junto con la necesidad de gestionar nodos, *launch files*, dependencias y comunicación entre procesos, introduce una sobrecarga significativa que no aporta ventajas reales en el contexto de un algoritmo que requiere miles de evaluaciones rápidas y ligeras de cinemática inversa. Por tanto, aunque estas herramientas son adecuadas para aplicaciones robóticas en tiempo real, su integración en el flujo de optimización planteado excede los objetivos y el alcance de la presente asignatura.

En consecuencia, se concluye que la disponibilidad de modelos cinemáticos adecuados y de herramientas software compatibles es un factor determinante para aprovechar al máximo el potencial de los algoritmos de optimización. La correcta codificación de los datos y la elección de las librerías apropiadas condicionan de manera directa la viabilidad computacional del enfoque y la calidad de las soluciones alcanzadas.

Definición multi-objetivo del problema y resolución

Fundamento técnico y justificación

Hasta este punto, el problema se ha abordado mediante una **función de mérito penalizada** que combina objetivos heterogéneos (tiempo máximo T_{max} , desincronización $|t_A - t_B|$ y violaciones de restricciones) en un único valor escalar. Según las recomendaciones técnicas recibidas, este enfoque presenta debilidades críticas: una alta sensibilidad a los pesos asignados y la posibilidad de generar soluciones artificiales por compensación numérica entre términos de distinta naturaleza física.

Por tanto, como se adelantó en la propuesta inicial, el problema del *rendezvous* robótico definido en este trabajo se puede formular como **bi-objetivo**. El objetivo ideal es minimizar simultáneamente los tiempos de ejecución de ambos robots:

$$\min_{\mathbf{p}_A, \mathbf{q}_A, \mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_A, \mathbf{s}_B, \mathbf{s}_{rail}} \mathbf{T}(\cdot), \quad (3)$$

con

$$\mathbf{T}(\cdot) = \begin{bmatrix} t_A(\mathbf{p}_A, \mathbf{q}_A, \mathbf{s}_A) \\ t_B(\mathbf{p}_B, \mathbf{q}_B, \mathbf{s}_B, \mathbf{s}_{rail}) \end{bmatrix},$$

sujeto a la restricción de sincronización $|t_A - t_B| \leq \Delta t$ y a las restricciones geométricas y cinemáticas del sistema. El planteamiento multiobjetivo permite obtener un **frente de Pareto**, proporcionando un conjunto de soluciones óptimas que revelan el compromiso real (*trade-off*) entre la velocidad de cada robot y la precisión de la sincronización, en lugar de una única solución condicionada por un ajuste manual de pesos.

Algoritmo seleccionado: NSGA-II

Para la resolución de este modelo se ha seleccionado el algoritmo **NSGA-II** (*Nondominated Sorting Genetic Algorithm II*). Esta técnica es el estándar para problemas con múltiples objetivos debido a su eficiencia y robustez. El algoritmo se basa en dos mecanismos fundamentales:

- **Ordenación No Dominada (*Nondominated Sorting*)**: Clasifica a los individuos en frentes sucesivos de dominancia de Pareto. Una solución domina a otra si es igual de buena en todos los objetivos y estrictamente mejor en al menos uno.
- **Distancia de Apiñamiento (*Crowding Distance*)**: Se utiliza para desempatar entre soluciones del mismo frente, favoreciendo aquellas situadas en regiones menos densas del espacio de objetivos para garantizar una distribución uniforme de soluciones a lo largo de todo el frente de Pareto.

Diseño y operadores

Siguiendo la arquitectura de dos niveles validada en el Tema 2, se mantiene una **codificación real** de las variables de decisión: $\mathbf{x} = (s_A, s_B, s_{rail}, p_{rail})$. Los operadores se adaptan de la siguiente manera:

- **Cruce y Mutación:** Se emplean operadores para codificación real compatibles con el sistema de cobots, manteniendo la reparación de soluciones (*InRange*) tras cada operación para asegurar la factibilidad cinemática.
- **Selección de Padres:** Se utiliza el **método de torneo**, que resulta más sencillo y eficaz en problemas multiobjetivo al permitir comparaciones directas basadas en la definición de optimalidad de Pareto.
- **Recombinación Elitista:** NSGA-II combina la población actual con la descendencia generada y selecciona los mejores individuos para la siguiente generación basándose en el frente de Pareto y la diversidad, asegurando que los mejores hallazgos no se pierdan.

Plan experimental y métricas de calidad

Siguiendo el rigor estadístico del **Anexo de evaluación**, se realizarán **30 ejecuciones independientes** para capturar la variabilidad del proceso [27, 198]. Dado que el resultado es un conjunto de soluciones (frente de Pareto) y no un valor único, se emplearán métricas específicas de optimización multiobjetivo [180, 199]:

1. **Hipervolumen (HV):** Mide el volumen del espacio de objetivos dominado por el frente obtenido respecto a un punto de referencia [180, 184].
2. **Indicador Epsilon (ϵ):** Evalúa la proximidad del frente hallado respecto a un frente de referencia ideal [180, 183].
3. **Diversidad y Cobertura:** Grado de extensión y uniformidad de las soluciones a lo largo de la frontera de Pareto [180, 199].

Resultados (Esquema)

Los resultados obtenidos mediante la aplicación de NSGA-II se presentan a continuación, comparando la calidad del frente de Pareto obtenido frente a los resultados mono-objetivo del algoritmo de Evolución Diferencial previo.

Configuración	Media HV	Desv. Típica HV	Mejor ϵ	Factibilidad (%)
NSGA-II (Propuesto)	–	–	–	–
DE Mono-objetivo	–	–	–	–

Tabla 6: Métricas de rendimiento multiobjetivo comparadas.

Figura 15: Visualización del Frente de Pareto obtenido por NSGA-II.



Referencias adicionales al Campus Virtual

- [1] UFactory. *xArm and 850 Robotic Arm*. Shenzhen UFactory Technology Co., Ltd. <https://www.ufactory.cc>. 2026.
- [2] Antonio Barrientos et al. *Fundamentos de robótica*. spa. 2.^a ed. Madrid: McGraw-Hill Interamericana de España, 2007. ISBN: 9788448156367.
- [3] Richard M Murray, Zexiang Li y S Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [4] Timothy D Barfoot y Paul T Furgale. "Associating Uncertainty With Three-Dimensional Poses for Use in Robotics". En: *IEEE Transactions on Robotics* 30.3 (2014), págs. 475-492.
- [5] Myles Hollander, Douglas A. Wolfe y Eric Chicken. *Nonparametric Statistical Methods*. 3.^a ed. John Wiley & Sons, 2013. DOI: 10.1002/9781119196037.